

Lecture 7:

Markov Chain Monte Carlo

September 16, 2026

anonymous course feedback:

<https://forms.gle/7ZSk2br7Z8AzznQw8>

some useful references

Machine Learning a Probabilistic Perspective, Kevin Murphy (**chapter 2**)

[https://raw.githubusercontent.com/kerasking/book-1/master/
ML%20Machine%20Learning-A%20Probabilistic%20Perspective.pdf](https://raw.githubusercontent.com/kerasking/book-1/master/ML%20Machine%20Learning-A%20Probabilistic%20Perspective.pdf)

Information Theory, Inference, and Learning Algorithms, David J.C. Mackay

<https://www.inference.org.uk/itprnn/book.pdf> (**chapter 2**)

office hours: Wed 9-10am

ask questions!

in-class: sample a Kroupa IMF using rejection sampling

$$\xi(m) \propto \begin{cases} (m/m_{\text{break}})^{-\alpha_1}, & m_{\text{min}} \leq m < m_{\text{break}} \\ (m/m_{\text{break}})^{-\alpha_2}, & m_{\text{break}} \leq m \leq m_{\text{max}} \end{cases}$$

$$\alpha_1 = 1.3, \alpha_2 = 2.3, m_{\text{break}} = 0.5 M_{\odot} \text{ sample over } [0.08, 100] M_{\odot}$$

rejection sampling

even with a clever choice of M q , a lot of samples are rejected
inefficient, especially in higher dimensions

importance sampling

If q is a poor match for p only a small fraction of samples will contribute to estimating $\mathbb{E}_p(f)$

importance weights $w = \frac{p}{q}$ can vary by large factors

Markov Chain Monte Carlo (MCMC)

from *SIAM News*, Volume 33, Number 4

The Best of the 20th Century: Editors Name Top 10 Algorithms

By Barry A. Cipra

Algos is the Greek word for pain. *Algor* is Latin, to be cold. Neither is the root for *algorithm*, which stems instead from al-Khwarizmi, the name of the ninth-century Arab scholar whose book *al-jabr wa'l muqabalah* devolved into today's high school algebra textbooks. Al-Khwarizmi stressed the importance of methodical procedures for solving problems. Were he around today, he'd no doubt be impressed by the advances in his eponymous approach.

Some of the very best algorithms of the computer age are highlighted in the January/February 2000 issue of *Computing in Science & Engineering*, a joint publication of the American Institute of Physics and the IEEE Computer Society. Guest editors Jack Dongarra of the University of Tennessee and Oak Ridge National Laboratory and Francis Sullivan of the Center for Computing Sciences at the Institute for Defense Analyses put together a list they call the "Top Ten Algorithms of the Century."

"We tried to assemble the 10 algorithms with the greatest influence on the development and practice of science and engineering in the 20th century," Dongarra and Sullivan write. As with any top-10 list, their selections—and non-selections—are bound to be controversial, they acknowledge. When it comes to picking the algorithmic best, there seems to be no best algorithm.

Without further ado, here's the CiSE top-10 list, in chronological order. (Dates and names associated with the algorithms should be read as first-order approximations. Most algorithms take shape over time, with many contributors.)

1946: John von Neumann, Stan Ulam, and Nick Metropolis, all at the Los Alamos Scientific Laboratory, cook up the Metropolis algorithm, also known as the **Monte Carlo method**.

The Metropolis algorithm aims to obtain approximate solutions to numerical problems with unmanageably many degrees of freedom and to combinatorial problems of factorial size, by mimicking a random process. Given the digital computer's reputation for

Markov Chain — a sequence of chains $x_0, x_1, \dots$ where the next state depends only on the current one

$$p(x_{t+1} \mid x_t, x_{t-1}, \dots, x_0) = p(x_{t+1} \mid x_t)$$

e.g. tomorrow's weather might depend on today's but not on last week's

Markov Chain Monte Carlo (MCMC) — the basic idea

Markov Chain Monte Carlo (MCMC) — the basic idea

to sample some distribution $p(x)$

you perform a random walk on the state space $\mathcal{X}^*$ in such a way that
the fraction of time spent in each state x is proportional to $p(x)$

**space of all possible outcomes that the random variable can take*

Markov Chain Monte Carlo (MCMC) — the basic idea

to sample some distribution $p(x)$

you perform a random walk on the state space $\mathcal{X}^*$ in such a way that
the fraction of time spent in each state x is proportional to $p(x)$

$$p(x) T(x' | x) = p(x') T(x | x')$$

$T(x' | x)$ is the transition probability — the chance of moving to x' given that you're at x

Markov Chain Monte Carlo (MCMC) — general algorithm

1. start from some state x
2. propose to move to new state x' with probability $q(x' | x)$
3. compute acceptance probability $r(x' | x)$
4. acceptance x' probability $r(x' | x)$; otherwise stay at x

repeat

Markov Chain Monte Carlo (MCMC) — general algorithm

1. start from some state x

2. propose to move to new state x' with probability $q(x' | x)$

3. compute acceptance probability $r(x' | x)$ $T(x' | x) = q(x' | x) r(x' | x)$

4. acceptance x' probability $r(x' | x)$; otherwise stay at x

repeat

Markov Chain Monte Carlo (MCMC) — Metropolis-Hasting

1. start from some state x

2. propose to move to new state x' with probability $q(x' | x)$

symmetric proposals (Metropolis): $q(x' | x) = q(x | x')$

3. compute acceptance probability $r(x' | x)$

$$r(x' | x) = \min (1, p(x')/p(x))$$

4. acceptance x' probability $r(x' | x)$; otherwise stay at x

repeat

$$p(x) T(x' | x) = p(x') T(x | x')$$

$$p(x) (q(x' | x) r(x' | x)) = p(x') (q(x | x') r(x | x'))$$

Markov Chain Monte Carlo (MCMC) — Metropolis-Hasting

1. start from some state x

2. propose to move to new state x' with probability $q(x' | x)$

asymmetric proposals with Hasting correction

$$a(x' | x) = \frac{p(x')}{p(x)} \frac{q(x | x')}{q(x' | x)}, \quad r(x' | x) = \min(1, a(x' | x))$$

$$p(x) T(x' | x) = p(x') T(x | x')$$

$$p(x) (q(x' | x) r(x' | x)) = p(x') (q(x | x') r(x | x'))$$

MCMC — Metropolis-Hasting algorithm to sample $p(x)$

MCMC — Metropolis-Hasting algorithm to sample $p(x)$

start from some state x

MCMC — Metropolis-Hasting algorithm to sample $p(x)$

start from some state x

propose to move to new state x' with probability $q(x' | x)$

MCMC — Metropolis-Hasting algorithm to sample $p(x)$

start from some state x

propose to move to new state x' with probability $q(x' | x)$

some proposal distribution

e.g. $\mathcal{N}(x' | x, \Sigma)$

MCMC — Metropolis-Hasting algorithm to sample $p(x)$

start from some state x

propose to move to new state x' with probability $q(x' | x)$

accept based on rate $r(x' | x) = \min \left(1, \frac{p(x') q(x | x')}{p(x) q(x' | x)} \right)$

MCMC — Metropolis-Hasting algorithm to sample $p(x)$

start from some state x

propose to move to new state x' with probability $q(x' | x)$

accept based on rate $r(x' | x) = \min \left(1, \frac{p(x') q(x | x')}{p(x) q(x' | x)} \right)$

ensures fraction of time spent is $\propto p(x)$

$$p(x) T(x' | x) = p(x') T(x | x')$$

MCMC — you don't need the evidence!

accept based on rate $r(x' | x) = \min \left(1, \frac{p(x') q(x | x')}{p(x) q(x' | x)} \right)$

only ever depends on the ratio so evidence cancels out

MCMC — Metropolis-Hasting algorithm

start from some state x

propose to move to new state x' with probability $q(x' | x)$

some proposal distribution

e.g. $\mathcal{N}(x' | x, \Sigma)$

MCMC — Metropolis-Hasting algorithm

start from some state x

propose to move to new state x' with probability $q(x' | x)$

some proposal distribution

e.g. $\mathcal{N}(x' | x, \Sigma)$

how do you select Σ ?

Σ too small = nearly every proposal gets accepted but each step barely moves

Σ too large = proposals often get rejected and your chain is standing still

MCMC — Metropolis-Hasting algorithm

start from some state x

propose to move to new state x' with probability $q(x' | x)$

some proposal distribution

e.g. $\mathcal{N}(x' | x, \Sigma)$

how do you select Σ ?

Σ too small = nearly every proposal gets accepted but each step barely moves

Σ too large = proposals often get rejected and your chain is standing still

for a **Gaussian target** in d -dimensions

optimal acceptance rate for MCMC is 23.4% (25-40% *in practice*)

in-class: implement MCMC using the Metropolis Algorithm
[ast384c/inclass/lec7_mcmc/Lec7_mcmc.ipynb](#)

challenges with Metropolis-Hasting algorithm

trade off between exploration and exploitation

(acceptance rate of 25-40% is theoretically optimal for a Gaussian target)

challenges with Metropolis-Hasting algorithm

trade off between exploration and exploitation

(acceptance rate of 25-40% is theoretically optimal for a Gaussian target)

initialization of the state $x = x_0$ — **burn-in** and **autocorrelation**

challenges with Metropolis-Hasting algorithm

trade off between exploration and exploitation

(acceptance rate of 25-40% is theoretically optimal for a Gaussian target)

initialization of the state $x = x_0$ — **burn-in** and **autocorrelation**

chain reflects where it started instead of the target distribution

challenges with Metropolis-Hasting algorithm

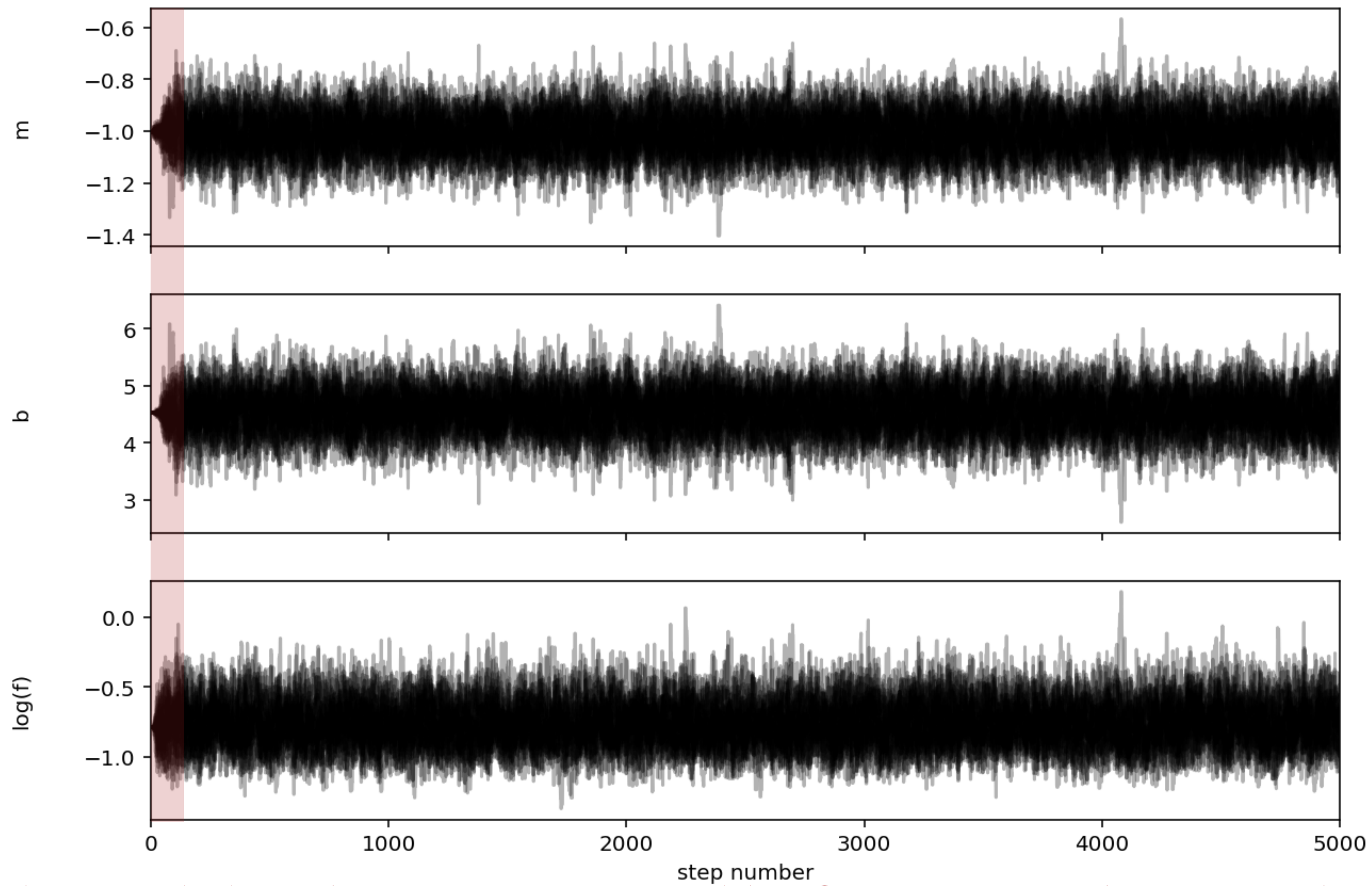
trade off between exploration and exploitation

(acceptance rate of 25-40% is theoretically optimal for a Gaussian target)

initialization of the state $x = x_0$ — **burn-in** and **autocorrelation**

chain reflects where it started instead of the target distribution

discard the “burn-in” period before using the samples



discard the “burn-in” period before using the samples

challenges with Metropolis-Hasting algorithm

trade off between exploration and exploitation

(acceptance rate of 25-40% is theoretically optimal for a Gaussian target)

initialization of the state $x = x_0$ — **burn-in** and **autocorrelation**

how long does the chain “remember” a previous state

sometimes the chain stays put so consecutive samples are correlated

challenges with Metropolis-Hasting — autocorrelation

$$\rho_f(\tau) = \frac{\text{Cov}(f(\theta_t), f(\theta_{t+\tau}))}{\text{Var}(f(\theta_t))}$$

$\tau = \text{lag}$
e.g. $f(\theta) = \theta$

$$\rho_f(0) = 1, \rho_f(\tau) \rightarrow 0 \text{ as } \tau \text{ grows}$$

challenges with Metropolis-Hasting — autocorrelation

$$\rho_f(\tau) = \frac{\text{Cov}(f(\theta_t), f(\theta_{t+\tau}))}{\text{Var}(f(\theta_t))}$$

integrated auto-correlation time: $\tau_f = \sum_{\tau=-\infty}^{\infty} \rho_f(\tau)$

challenges with Metropolis-Hasting — autocorrelation

$$\rho_f(\tau) = \frac{\text{Cov}(f(\theta_t), f(\theta_{t+\tau}))}{\text{Var}(f(\theta_t))}$$

integrated auto-correlation time: $\tau_f = \sum_{\tau=-\infty}^{\infty} \rho_f(\tau)$

correlated MCMC samples make Monte Carlo integrals less accurate —
error shrinks as $1/\sqrt{N/\tau_f}$, not $1/\sqrt{N}$

challenges with Metropolis-Hasting — autocorrelation

$$\rho_f(\tau) = \frac{\text{Cov}(f(\theta_t), f(\theta_{t+\tau}))}{\text{Var}(f(\theta_t))}$$

integrated auto-correlation time: $\tau_f = \sum_{\tau=-\infty}^{\infty} \rho_f(\tau)$

correlated MCMC samples make Monte Carlo integrals less accurate —

error shrinks as $1/\sqrt{N/\tau_f}$, not $1/\sqrt{N}$

“effective sample size”

Goodman & Weare (2010): *Ensemble samplers with Affine Invariance*

Goodman & Weare (2010): *Ensemble samplers with Affine Invariance*

multiple concurrent chains (or “walkers”)

Goodman & Weare (2010): *Ensemble samplers with Affine Invariance*

multiple concurrent chains (or “walkers”)

walkers inform each other of their acceptance rates for efficiency

Goodman & Weare (2010): *Ensemble samplers with Affine Invariance*

multiple concurrent chains (or “walkers”)

walkers inform each other of their acceptance rates for efficiency

tune Σ based on this information

Goodman & Weare (2010): *Ensemble samplers with Affine Invariance*

multiple concurrent chains (or “walkers”)

walkers inform each other of their acceptance rates for efficiency

an ensemble of N walkers can converge faster
than a single with N times the steps

Goodman & Weare (2010): *Ensemble samplers with Affine Invariance*

multiple concurrent chains (or “walkers”)

walkers inform each other of their acceptance rates for efficiency



How long should I run my MCMC chains?

rule of thumb: chain length N should be *at least* $50\tau_f - 1000\tau_f$ for
high confidence

emcee's guideline: keep sampling until $N > 50\tau$ **and**
your τ estimate converges to $< 1\%$

a typical arbitrary emcee run

walkers: ~30

(a small multiple of the number of parameters)

iterations: ~5000

burn-in: ~100

thinning: keep 1 in 15

in-class: run MCMC using emcee

[ast384c/inclass/lec7_mcmc/Lec7_emcee.ipynb](#)

MCMC is the standard sampling algorithm in Bayesian inference, but what about *very* high dimensional distribution

Hamiltonian Monte Carlo